

Sisteme de operare

Laborator 13

Semestrul I 2025-2026

Laborator 13

- advanced I/O
- programare client-server TCP/IP
 - servere concurente TCP/UDP
 - servere multiprotocol
 - superserverul internet (inetd)

Multiplexarea I/O

- Q: cum citim date din doi descriptori de fisiere diferiti cand nu stim cand vor fi disponibile datele?
 - obs: citirea blocanta cu *read* nu functioneaza
- solutii posibile
 - *polling* (setam operatii nebloccante pe descriptorii de fisiere si citim in bucla pe rand)
 - dezavantaje: proasta utilizare a CPU (putem folosi *sleep* pentru a ameliora situatia, dar nu stim nici cat sa asteptam !)
 - *I/O asincron*
 - instruim kernelul sa ne trimita un semnal (SIGPOLL/SIGIO) cand datele sunt disponibile
 - dezavantaj: exista un singur asemenea semnal per proces
 - => la livrarea semnalului nu stim de pe care descriptor sa citim
 - trebuie sa facem descriptorii non-blocanti si sa incercam citirea in secventa (*polling*)

Multiplexarea I/O (cont.)

- Q: cum citim date din doi descriptori de fisiere diferiti cand nu stim cand vor fi disponibile datele?
 - obs: citirea blocanta cu *read* nu functioneaza
- solutii posibile
 - ...
 - *multiplexarea I/O*
 - definim o lista de descriptori de fisiere de care suntem interesati
 - apelam o functie care se deblocheaza doar cand exista descriptori disponibili pt operatii de I/O (exista date disponibile, de ex)
 - dupa deblocare, exista functii care ne ajuta sa identificam acesti descriptori
 - ex: *select*, *poll*

select

- permite sa notificam kernelul in privinta
 - descriptorilor de interes
 - conditiilor care ne intereseaza: *read/write/exceptii*
 - timpului de asteptare a disponibilitatii de I/O (deloc, pentru totdeauna sau pt o perioada specificata de timp)
- kernelul raspunde cu urmatoarele informatii
 - nr total de descriptori disponibili pt operatii de I/O
 - care descriptori sunt disponibili pentru fiecare tip de conditie: *read/write/exceptie*

```
int select(int nfds, fd_set *readfds, fd_set *writefds,  
          fd_set *exceptfds, struct timeval *timeout);
```

Valori timeout

- *NULL* => se asteapta la infinit
 - *select* se deblocheaza cand un descriptor e gata de I/O sau la primirea unui semnal
 - la primirea unui semnal *select* intoarce -1 si `errno = EINTR`
- *timeout.tv_sec == 0 && timeout.tv_usec == 0*
 - *select* se intoarce imediat, fara blocare
 - se foloseste in *polling*
- *timeout.tv_sec != 0 || timeout.tv_usec != 0*
 - asteapta durata de timp specificata in secunde si microsecunde
 - *select* se deblocheaza
 - cand un descriptor a devenit disponibil
 - cand a expirat perioada de timp
 - la primirea unui semnal
 - daca timeout-ul a expirat si niciun descriptor n-a devenit disponibil, *select* intoarce valoarea 0

Seturi de descriptori

- *readfds*, *writefds*, *exceptfds* specifica descriptorii de interes pt operatiile de I/O corespunzatoare
- *fd_set* = tip de date *bitmap*, 1 bit per descriptor de interes
- operatii cu seturi de descriptori
 - alocarea unei variabile
 - asignarea de variabile
 - operatii pe bitmap-uri

*FD_ZERO(fd_set *fds);* // pune bitii din *fds* pe 0

*FD_SET(int fd, fd_set *fds);* // pune pe 1 bitul *fd* din *fds*

*FD_CLR(int fd, fd_set *fds);* // pune pe 0 bitul *fd* din *fds*

*FD_ISSET(int fd, fd_set *fds);* // testeaza daca bitul *fd* din *fds* e setat

Seturi de descriptori (cont.)

- secventa tipica de operare

```
fd_set rfds;
```

```
int fd;
```

```
...
```

```
FD_ZERO(&rfds);
```

```
FD_SET(fd, &rfds);
```

```
if(select(nfds, &rfds, ....) < 0)
```

```
    exit(1);
```

```
if(FD_ISSET(fd, &rfds))
```

```
    read(fd, buf, sizeof(buf) - 1); // read nebloant !
```

Nr max de descriptori si valoare de retur

- cand nu stim valorile descriptorilor
 - primul parametru a lui *select* se alege ca fiind nr maxim de file descriptori posibil (FD_SETSIZE sau apel biblioteca *getdtablesize*)
- daca stim valoarea maxima a descriptorilor pe care ii folosim
 - o folosim ca valoare a primului argument de apel al lui *select*
 - pt ca FD_SETSIZE poate fi mare (eg 1024, verificati cu *ulimit -n*) => optimizare a cautarilor kernelului pt descriptori disponibili
- *select* intoarce:
 - -1 in caz de eroare
 - 0 pt. timeout expirat si niciun descriptor disponibil
 - > 0 : nr de descriptori disponibili pt operatii de I/O
 - cei pt care operatiile de R/W nu se vor bloca
 - respectiv, descriptorii pt care exista o conditie de exceptie in asteptare (eg, OOB data in conexiuni de retea)

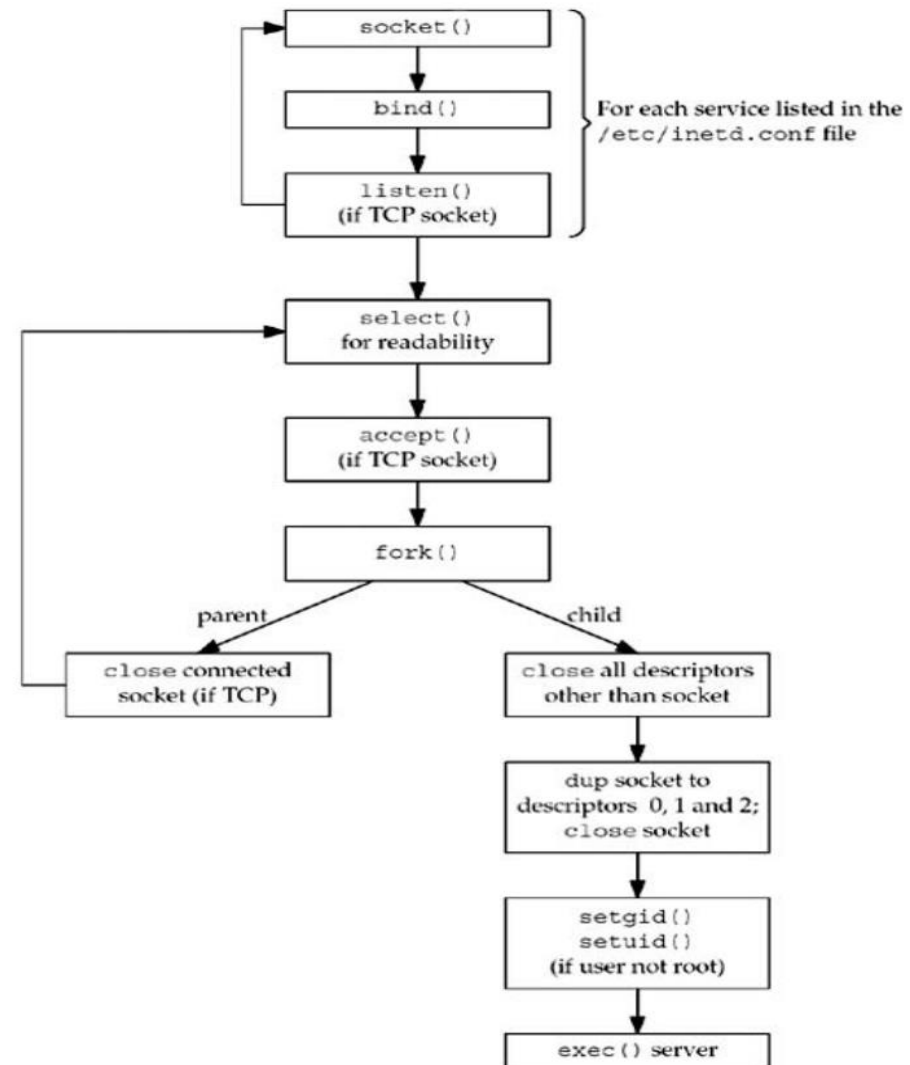
Inetd

- superserverul internet
 - multiplexează servicii multiple
 - proces părinte care rulează *neintrerupt* și *nemodificat*
 - fiecare cerere client este tratată separat de către un proces copil
 - procesele copil apelează *exec* cu un program server specific pt fiecare serviciu
- programul server
 - nu gestionează direct socket-uri
 - toată activitatea de I/O este desfășurată pe *stdin/stdout*
- legătura dintre *inetd* (procesul părinte) și programele server (procesele copil)
 - înainte de apelul *exec* se duplică socket-ul de serviciu în descriptorii 0,1,2

Inetd (cont.)

- avantaj:
 - dezvoltare de cod simplificata
 - *inetd* ruleaza ***nemodificat in permanenta***
 - programele server se pot recompila oricand pentru a adauga functionalitate, bug fixing, etc.
 - tot ce face *inetd* este sa apeleze un program executabil cu o cale bine specificata
- dezavantaj: performanta (eg, v. discutia *fork-exec* vs thread-uri)

Structura server inetd



Sisteme de operare

Laborator 13

Servere concurente si multiprotocol TCP/UDP, superserverul Internet (inetd)

1. Creati un nou subdirector *lab13/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab13/bin*.

Pentru a simplifica scrierea codului puteti folosi urmatoarele functii de biblioteca furnizate in **libsock.a**:

```
int connectTCP(const char *host, const char *service);  
int connectUDP(const char *host, const char *service);
```

Cele doua functii de mai sus creeaza un socket activ si conectat (**N.B.** in cazul UDP, conectarea nu inseamna decat ca puteti folosi apelurile sistem *read/write* pentru a lucra cu sockets). Primul parametru reprezinta host-ul la care se conecteaza socket-ul activ, iar cel de-al doilea reprezinta serviciul identificat prin numele sau canonic listat in fisierul */etc/services*. Valoarea de retur este file descriptorul care reprezinta socket-ul activ sau -1 in caz de eroare. Exemple de apel:

```
int sd = connectTCP("localhost", "time");  
int sd = connectUDP("127.0.0.1", "daytime");
```

Un socket pasiv se poate crea folosind unul dintre urmatoarele apeluri:

```
int passiveTCP(const char *service, int qlen);  
int passiveUDP(const char *service);
```

unde, in mod similar cu apelurile client, valoarea primului parametru reprezinta serviciul pe care il executa serverul iar valoarea de retur reprezinta socket descriptorul de serviciu. Al doilea parametru de apel pentru *passiveTCP* reprezinta valoarea de *backlog*. Exemple de apel:

```
int sd = passiveTCP("time", 128);  
int sd = passiveUDP("time");
```

Pentru a putea folosi functiile de mai sus in programele pe care le scrieti e necesar sa linkeditati biblioteca **libsock.a** ca mai jos, de exemplu:

```
$ gcc -o timed-udp timed-udp.c -L. -lsock
```

2. Modificati serverul **timed-udp.c** pt serviciul *time* din laboratorul trecut pentru a deveni un *server multiprotocol*, care raspunde atat la cereri TCP cat si UDP din partea clientilor.

Indicatie: folositi *select*.

3. Modificati serverul **echod-tcp.c** pentru serviciul *echo* TCP scris in laboratorul trecut a.i. sa devina un *server concurent* (i.e., sa fie capabil sa raspunda la cereri concurente ale clientilor). In acest scop, folositi apelul sistem *select*.

4. Scrieti un server TCP concurent **echod-tcp.c** pentru serviciul de *echo* care sa raspunda la cererile clientului *echo* TCP din laboratorul trecut. Pentru fiecare cerere client, serverul (proces master) creeaza un nou proces (proces copil, slave) folosind apelul sistem *fork* care va trata separat cererea client, in vreme ce serverul parinte (master) revine la activitatea de acceptare a cererilor clientilor.

Obs: Pentru a opera in mod concurent, serverul parinte nu poate astepta in mod blocant terminarea proceselor (copii) care trateaza cererile client. Cum procedati pentru ca procesele copil sa nu ramana in starea de zombie?

5. Folositi shell script-ul din laboratorul trecut pentru a verifica functionalitatea concurenta a serverelor **echod** de mai sus (exercitiile 3 si 4).

6. Implementati propria versiune a apelului de biblioteca *usleep*, care suspenda executia programului pentru un numar de microsecunde dat:

```
int my_usleep(unsigned int usecs);
```

Apoi scrieti un scurt program in care sa verificati functia implementata.

Indicatie: folositi *select*.

7. Scrieti programe server TCP **echo-inetd.c**, **daytime-inetd.c** si **time-inetd.c** pentru serviciile *echo*, *daytime* si *time* respectiv, care sunt apelabile de catre superserverul internet **inetd**. Concret, aceste programe nu trebuie sa creeze nici un socket. Ele citesc de la *stdin* o cerere venita de la client si trimit raspunsul catre client la *stdout* (pe ecran).

Obs: programele server se ocupa doar de o singura cerere client, **inetd** gestioneaza cereri multiple.

8. Scrieti un superserver Internet **inetd.c** care accepta cereri de la clienti TCP pentru serviciile de *echo*, *daytime* si *time* si le proceseaza folosind programele server de la exercitiul anterior.

Indicatie: serverul **inetd** va crea sockets pentru cele trei servicii si ii va inregistra intr-un file descriptor set pe care il va pasa ca parametru apelului *select*. Cand unul dintre descriptorii de fisier devine disponibil pt citire (i.e., un client a facut o cerere), **inetd** apeleaza *accept* si paseaza noul socket creat unui proces copil creat cu *fork* care va executa serviciul corespunzator cererii invocand apelul sistem *exec* (cu variantele sale) caruia ii paseaza ca parametru programul corespunzator dezvoltat la exercitiul anterior.